# 《面向对象程序设计》实验讲义总体说明

《面向对象程序设计》实验讲义是《面向对象程序设计》实验课程的教学指导，该实验课程共由 8 个实验组成，每个实验的内容和要求在下面给出，这 8 个实验的总成绩为 100 分，折合成《面向对象程序设计》课程期末总成绩的 20 分。

## 实验一 类与对象（2 学时）

### 一、实验目的

1．掌握类的定义和对象的声明；

2．掌握具有不同访问属性的成员的访问方式；

3．掌握构造函数和析构函数的编写方法；

4．掌握类的组合的使用方法。

### 二、实验内容

1．设计一个 Tree （树）类，有成员 ages （树龄），成员函数 grow(int years) ages 加上 years，age( )显示 tree 对象的 ages 的值。

2．设计一个用于人事管理的 People 类。考虑到通用性，可以只抽象出所有人员都具有的属性：num(编号)，name(姓名) ，birth(生日)，id(身份证号)等等。其中“生日”定义为一个“日期”类内嵌子对象。用成员函数实现对人员的信息录入和显示。

### 三、实验指导

题目 1

（1）定义 Tree 类的框架

（2）在 Tree 类中加入一个数据成员 ages，类型为整型。【访问权限同学们自己思考设计】

（3）在 Tree 类中加入成员函数：【访问权限同学们自己思考设计】

①Tree 类的默认构造函数：实现为 Tree 类对象的 ages 成员初始化

②grow(int years) :为某个 Tree 类的对象的 ages 增加 years 年

③age( ):显示 Tree 类对象的 ages

（4）编写 main( )函数，在其中建立两个 Tree 类的对象，分别测试该类中的所有成员函数。

题目 2

（1）编写 Date（日期）类，Date 类中包括 3 个数据成员(year,mon,day)以及如下成员函数：

①构造函数【同学们自己设计构造函数的个数和形式】

②复制构造函数

③修改日期的函数

④显示日期的函数

⑤析构函数

（2）编写 People 类，People 类中包括 4 个数据成员（num, name, id, birth），其中 birth 为 Date 类的对象；以及如下成员函数：

①构造函数【同学们自己设计构造函数的个数和形式】

②复制构造函数

③信息录入函数：实现对某个 People 类对象的信息录入

④信息显示函数:实现对某个 People 类对象的信息显示

⑤析构函数

（3）编写 main( )函数，在其中建立两个 Date 类的对象和两个 People 类的对象，分别测试这两类中的所有成员函数。

**四、考核标准**

本实验总计 15 分。

1、题目 1 总计 5 分

（1） 类的设计与编写 （3 分）

（2） 程序功能，运行结果的正确性（2 分）

2、题目 2 总计 10 分

（1） 友好的用户界面 （2 分）

（2） Date 类的编写 （2 分）

（3） People 类的编写，要求实现类的组合机制 （4 分）

（4） 程序功能，运行结果的正确性 （2 分）

## 实验二 C++程序的结构（2 学时）

### 一、实验目的

1．掌握程序运行中变量的作用域、生存期和可见性；

2．掌握类的静态成员和友元函数的使用；

3．掌握使用工程和多文件结构编写程序的方法。

### 二、实验内容

1．设计并编写 Songer（歌手）类，使用静态数据成员和静态成员函数实现统计当前歌手的数量，使用友元函数实现任意两个歌手之间的 Pk。

2．将实验一中第 2 题的内容使用工程和多文件结构实现。

### 三、实验指导

题目 1

（5）Songer 类包含如下数据成员：

①name （姓名）

②fans_count （粉丝数）

③count　　（当前歌手人数）　【静态数据成员】

（6）在 Songer 类中加入成员函数：

①构造函数：初始化 name 和 fans_count，并对 count 做适当操作

②复制构造函数：【同学们自己设计并编写】

③Set 函数：输入 fans_count 的值（即修改粉丝数）

④析构函数 【同学们自己设计并编写】

⑤ShowCount 函数：显示当前歌手的数量。【静态成员函数】

（7）编写 Songer 类的友元函数：Pk（Songer &s1,Songer &s2），实现比较两个 Songer 类对象的 fans_count 并输出结果。

【例如：如果 s1 的姓名是张三，有 100 个粉丝；s2 的姓名是李四，有 120 个粉丝，输出李四战胜张三】

（8）设计编写 main()函数，在其中建立三个 Songer 类的对象，测试该类中所有成员函数和友元函数。

*注：学有余力的同学可以广开脑洞，在完成题目要求基础上设计并编写合理功能。*

题目 2：【根据课堂学习，熟练掌握建立工程以及使用多文件编写程序的方法】

**四、考核标准**

本实验总计 10 分。

1、题目 1 总计 5 分

（3） 类的设计与编写　　（2 分）

（4） 静态成员以及友元函数的编写（2 分）

（5） 程序功能，运行结果的正确性（1 分）

2、题目 2 总计 5 分

（5） 程序结构框架的合理性以及操作熟练性 （3 分）

（6） 程序功能，运行结果的正确性 （2 分）

# 实验三 C++中的动态内存分配（2 学时）

## 一、实验目的

1．掌握使用 new 运算符创建对象的方法；

2．掌握使用 delete 运算符释放对象内存的方法；

3．掌握使用面向对象思想实现链表类并对链表进行操作的方法。

## 二、实验内容

1．设计并编写链表类，能够实现链表的初始化（建立）、输出释放等功能。

## 三、实验指导

（1）链表由若干个结点【Node 类对象】构成（每个结点有 data 域和 next 域，假设每个结点的 data 域是一个整数，并且没有重复元素），那么某个链表的逻辑示意图如下：

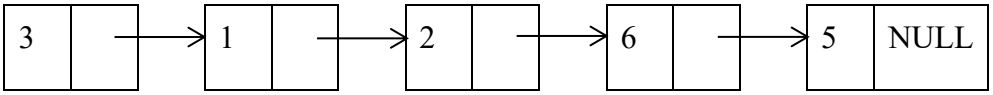


图 3-1 链表逻辑结构示意图

下面给出链表中结点类的定义：

```
class Node    //结点类
{     public:
          Node( );                          //构造函数
          void SetNext(Node *p);            //设置后继结点
          int Getd();                       //得到当前结点的数据值
          Node *GetNext( );                 //得到后继结点的地址
          ~Node( );                         //析构函数
      private:
          int       data;
          Node    *next;
```

```
};
Node::Node( )
{ cin>>data; next=NULL;}
void Node::SetNext(Node *p)
{ next=p;}
int Node::Getd( )
{ return data;}
Node* Node::GetNext( )
{ return next;}
Node::~Node( )
{ cout<<data<<"  被析构！"<<endl;}
```

**上面的 Node 类代码同学们可以直接使用，下面（2）、（3）为必做内容**

（2）利用 Node 类，设计并编写 Link（链表）类，类中包含：

- 数据成员
  - Head　//头指针
  - Num　//链表中结点个数 【即链表的长度】
- 函数成员：
  - Link( )　//初始化（建立）空链表
  - Link(int n)　//初始化（建立）一个具有 n 个结点的链表

    【在建立链表的过程中按值从小到大排序】
  - LinkPrint( )　//输出链表中所有结点 data 域的值
  - GetHead( )　//返回链表的头指针

- ~Link( )　　//释放链表中所有结点的内存空间

（3）编写 main( )函数，测试 Link 类中所有函数

【选作内容：编写 Link 类的如下成员函数，根据实际完成情况有加分】

- Link_Insert(int x)　　//在链表中插入值为 x 的结点
- Link_Delete( int x)　　//删除链表中值为 x 的结点
- Link_Connect(Link　&p)　　//两个链表的连接

  【即将 p 链表连接在当前链表的尾部】
- Link( Link　&p)　　//使用深复制编写复制构造函数
- 自己设计 Node 类中 data 的类型，使本题实现一个具体的应用

**四、考核标准**

本实验总计 10 分。

（6） Link 类的框架 （2 分）

（7） Link 类成员函数的实现（5 分）

（8） main（）函数编写的合理性及功能演示（3 分）

## 实验四 继承和派生（2 学时）

**一、实验目的**

1．掌握利用继承和派生机制定义并实现派生类的方法；

2．掌握派生类构造函数的编写方法；

3、掌握测试具有继承结构的类家族中各类对象的方法。

**二、实验内容**

1．设计并编写 Person（人员）类、Student（学生）类和 Graduate（研究生）类，并测试这些类。

**三、实验指导**

1、类之间的和关系：

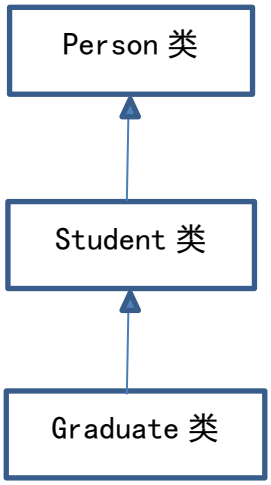


图 4-1 各个类继承结构示意图

2、具体内容

**（1）设计并编写 Person 类，包括：**

- 数据成员

  ① name　//姓名

  ② sex　//性别

  ③ age　//年龄

- 函数成员

  ①默认（无参）构造函数

  ②有参构造函数

  ③复制构造函数

  ④设置信息函数

  ⑤显示信息函数

**（2）Person 类派生出 Student 类，Student 类中添加：**

- 数据成员

  ① Num　　//学号

  ② School　　//学校

  ③ Major　　//专业

- 函数成员

  ①默认（无参）构造函数

  ②有参构造函数

  ③复制构造函数　　【③为选做】

  ④设置信息函数

  ⑤显示信息函数

**（3）Student 类派生出 Graduate 类，Graduate 类中添加：**

- 数据成员

  ① Research_Direction　　//研究方向

- 函数成员

  ①默认（无参）构造函数

  ②有参构造函数

  ③复制构造函数　　【③为选做】

  ④设置信息函数

  ⑤显示信息函数

**（4）设计并编写 main 函数，测试编写的类。**

**四、考核标准**

本实验总计 10 分。

（9） Person 类的设计与实现 （2 分）

（10）Student 类的设计与实现 （3 分）

（11）Graduate 类的设计与实现 （3 分）

（12）main（）函数编写的合理性及功能演示（2 分）

## 实验五 运算符重载（2 学时）

### 一、实验目的

1．掌握运算符重载的作用；

2．掌握运算符重载为成员函数的方法；

3. 掌握运算符重载为非成员函数的方法

### 二、实验内容

1．为实验二中编写的 Songer（歌手）类重载“+”、“>”、“==”、前置“++”、后置“++”等运算符，使 Song 类的功能更丰富。

### 三、实验指导（其中 1-5 为必做，6 为选做）

1. 重载“+”运算符,实现如下功能

**例：**s1 是 Songer 类的对象，则 s1+100 能够实现将 s1 的粉丝数加 100

2. 将“>”运算符重载为成员函数,实现如下功能

**例：**s1、s2 是 Songer 类的对象，则 s1>s2 能够实现比较 s1 的粉丝数是否多于 s2 的粉丝数

3. 将“==”运算符重载为友元函数,实现如下功能

**例：**如 s1、s2 是 Songer 类的对象，则 s1==s2 能够实现比较 s1 的粉丝数是否等于 s2 的粉丝数

4. 将前置“++”运算符重载为成员函数，实现如下功能

**例：**s1 是 Songer 类的对象，则 ++s1 能够实现将 s1 的粉丝数加 10

5. 将后置“++”运算符重载为友元函数，实现如下功能

**例：**s1 是 Songer 类的对象，则 s1++ 能够实现将 s1 的粉丝数加 10

6.【选做】：重载“<<”运算符，实现如下功能

**例：**s1 是 Songer 类的对象，则 cout<<s1 能够实现输出 s1 的信息（姓名、粉丝数）

**四、考核标准**

本实验总计 10 分。

（13）各个重载运算符的编写　　（6 分）

（14）main 函数测试这些运算符的合理性及正确性　（4 分）

## 实验六 虚函数（2 学时）

**一、实验目的**

1．掌握作用的作用；

2．掌握纯虚函数的使用；

3． 掌握使用虚函数实现动态多态性。

**二、实验内容**

1．树结构是一类重要的非线性数据结构。树结构在客观世界中广泛存在，如人类社会的族谱和各种社会组织机构都可以用树形象表示，树在计算机领域中也得到广泛应用，尤以二叉树最为常用。

2. 有以下 Tree（树）、Binary_Tree（二叉树）和 Binary_Sort_Tree（二叉排序树）类的定义，其中 Binary_Tree 继承了 Tree，Binary_Sort_Tree 继承了 Binary_Tree。

```
class Tree//树
{
  public:
      virtual void show();
      virtual ~Tree();
};
```

```
Tree::~Tree()
{
   cout<<"析构一个树对象!"<<endl;
}
void Tree::show()
{
   cout<<"树是非常重要的一种数据结构！"<<endl;
}
class Binary_Tree:public Tree//二叉树
{
          public:
               virtual void show();
               virtual ~Binary_Tree();
};

Binary_Tree::~Binary_Tree()
{
   cout<<"析构一个二叉树对象!"<<endl;
}

void Binary_Tree::show()
{
       cout<<"二叉树每个结点至多有两棵子树，且有左右之分！"<<endl;
}
class Binary_Sort_Tree:public Binary_Tree//二叉排序树
{
         public:
```

```
        virtual void show();
        virtual ~Binary_Sort_Tree();
};

Binary_Sort_Tree::~Binary_Sort_Tree()
{
    cout<<"析构一个二叉排序树对象!"<<endl;
}
void Binary_Sort_Tree::show()
{
    cout<<"二叉排序树首先是一颗二叉树！"<<endl;
    cout<<"如果左子树不空，左子树小于根节点！"<<endl;
    cout<<"如果右子树不空，右子树大于根节点！"<<endl;
}
```

**三、实验指导**

1. 定义一个 Tree 的引用做参数的函数，在函数中访问 show（）函数，通过三种不同的对象来调用该函数以实现多态性。

6. 定义一个 Tree 的指针做参数的函数，在函数中访问 show（）函数，通过三种不同对象的地址来调用该函数以实现多态性。

7. 定义一个 Tree 类的指针，通过 new 操作产生不同的对象，然后 delete 该指针，观察虚析构实现的多态性。

8. （选做）如果将 Tree 类中 show 函数改为纯虚函数，在（3）中会出现什么情况？

**四、考核标准**

本实验总计 10 分。

（15） （3 分）

（16） （3 分）

（17） （4 分）

# 实验七 群体类（2 学时）

## 一、实验目的

1．掌握函数模板的编写和使用方法；

2．掌握类模板的编写和使用方法。

## 二、实验内容

1．编写一个用选择法给数组排序的函数模板。

2．编写一个集合类模板。

## 三、实验指导

1、编写一个用选择法给数组排序的函数模板，函数模板为 mySort(T a[],int n) 实现给 n 个 T 类型数据排序，并在主函数中测试。

2、编写一个集合类模板 Set（可以参照教材中 Array 类模板），Set 可以包含零个或多个无序的非重复元素。具体内容如下：

（1）集合类模板 Set

```
template <class T>
class Set
{
    private:
        T*    value;    //T 类型指针，用于存放集合的元素的动态数组，长度为 length
        int size;       //集合大小（元素个数）
        int   length;   //动态数组长度
    public:
        Set(int len = 50);     //构造函数
        Set(Set<T>& A);    //拷贝构造函数 （选做）
```

```
    ~Set( );      //析构函数
    Set<T>& operator= (Set<T>& r);   //重载"="使集合对象可以整体赋值
    Set<T>   operator+ (Set<T>& r);   //重载"+"求两个集合对象的并集
    void display( );   //输出集合中所有元素
    Bool isExist(T v); //判断集合中是否包含元素 v
    void add(T v);     //向集合中添加一个元素
    int getSize( )   ;   //  返回集合的大小
    int getLength( );   //  返回动态数组 value 的大小
    void Resize(int len); //  修改动态数组 value 的大小
};
```

（2）**设计并编写 main 函数，测试编写的类。**

**四、考核标准**

本实验总计 10 分。

（18）函数模板为 mySort 的设计与实现 （4 分）

（19）Set 类模板的设计与实现 （6 分）

## 实验八 流类库（2 学时）

**一、实验目的**

1．掌握流类库中常用的类及其成员函数的用法；

2．掌握使用标准输入输出流进行文件的操作的方法（二进制、文本）。

**二、实验内容**

设计学生类，及学生信息控制类。要求能实现学生的信息的增加、修改、删除与查询。

**三、实验指导**

**（1）设计并编写学生类 Student，包括：**

- 数据成员

  ① Num　　//学号

  ② name　　//姓名

  ③ sex　　//性别

  ④ score　//成绩

- 函数成员

  ①默认（无参）构造函数

  ②有参构造函数

  ③复制构造函数

  ④设置信息函数

  ⑤显示信息函数

**（5）StudentControl 类：**

- 数据成员

  ① 定义一个存储若干学生的数据成员，可以在下面几种方法中选择：

  A. list<Student>　listStu ;//链表容器　　有加 5 分

  B. vector<Student>　vecStu（10）;//向量容器　有加 5 分

  C. Student　* stuP;　　//根据需要确定数组容量，要动态地变化

  D. Student　stu[100]; //存储学生信息的数组

  ② int num; //学生个数

- 成员函数

  ①构造函数

  ②loadFromFile(char *fielname);//从文件中读取学生信息存储到第一个

数据成员；

③display();//显示所有第一个数据成员中存储学生信息；

④input(); //键盘输入若干学生信息，存储到第一个数据成员；

⑤sort(); //对学生信息按成绩排序；

⑥writeToFile(char *fielname);//把第一个数据成员中的学生信息写入文件

⑦析构函数

**（6）设计并编写 main 函数，测试编写的类。**

**四、考核标准**

本实验总计 10 分。

（20）Student 类的设计与实现 （2 分）

（21）**StudentControl** 的设计与实现 （5 分）

（22）main()函数编写的合理性及功能演示（3 分）

如果采用 list 存储数据，可参考下面成员函数。

参考文献：

链表 list 将元素按顺序储存在链表中. 与 向量(vector)相比, 它允许快速的插入和删除，但是随机访问却比较慢.

list 的成员函数：

assign() 给 list 赋值

back() 返回最后一个元素

begin() 返回指向第一个元素的迭代器

clear() 删除所有元素

empty() 如果 list 是空的则返回 true

end() 返回末尾的迭代器

erase() 删除一个元素

front() 返回第一个元素

get_allocator() 返回 list 的配置器

insert() 插入一个元素到 list 中

max_size() 返回 list 能容纳的最大元素数量

merge() 合并两个 list

pop_back() 删除最后一个元素

pop_front() 删除第一个元素

push_back() 在 list 的末尾添加一个元素

push_front() 在 list 的头部添加一个元素

rbegin() 返回指向第一个元素的逆向迭代器

remove() 从 list 删除元素

remove_if() 按指定条件删除元素

rend() 指向 list 末尾的逆向迭代器

resize() 改变 list 的大小

reverse() 把 list 的元素倒转

size() 返回 list 中的元素个数

sort() 给 list 排序

splice() 合并两个 list

swap() 交换两个 list

unique() 删除 list 中重复的元素

实例 1：

```
#include <iostream>
#include <list>
#include <numeric>
#include <algorithm>
using namespace std;

//创建一个 list 容器的实例 LISTINT
typedef list<int> LISTINT;
//创建一个 list 容器的实例 LISTCHAR
typedef list<int> LISTCHAR;

void main()
{
    //用 list 容器处理整型数据
    //用 LISTINT 创建一个名为 listOne 的 list 对象
    LISTINT listOne;
    //声明 i 为迭代器
    LISTINT::iterator i;

    //从前面向 listOne 容器中添加数据
    listOne.push_front (2);
    listOne.push_front (1);

```

```
//从后面向 listOne 容器中添加数据
listOne.push_back (3);
listOne.push_back (4);

//从前向后显示 listOne 中的数据
cout<<"listOne.begin()--- listOne.end():"<<endl;
for (i = listOne.begin(); i != listOne.end(); ++i)
    cout << *i << " ";
cout << endl;

//从后向后显示 listOne 中的数据
LISTINT::reverse_iterator ir;
cout<<"listOne.rbegin()---listOne.rend():"<<endl;
for (ir =listOne.rbegin(); ir!=listOne.rend();ir++) {
    cout << *ir << " ";
}
cout << endl;

//使用 STL 的 accumulate(累加)算法
int result = accumulate(listOne.begin(), listOne.end(),0);
cout<<"Sum="<<result<<endl;
cout<<"------------------"<<endl;

```

```
//-------------------------
//用 list 容器处理字符型数据
//-------------------------

//用 LISTCHAR 创建一个名为 listOne 的 list 对象
LISTCHAR listTwo;
//声明 i 为迭代器
LISTCHAR::iterator j;

//从前面向 listTwo 容器中添加数据
listTwo.push_front ('A');
listTwo.push_front ('B');

//从后面向 listTwo 容器中添加数据
listTwo.push_back ('x');
listTwo.push_back ('y');

//从前向后显示 listTwo 中的数据
cout<<"listTwo.begin()---listTwo.end():"<<endl;
for (j = listTwo.begin(); j != listTwo.end(); ++j)
    cout << char(*j) << " ";
cout << endl;

```

```
//使用 STL 的 max_element 算法求 listTwo 中的最大元素并显示
j=max_element(listTwo.begin(),listTwo.end());
cout << "The maximum element in listTwo is: "<<char(*j)<<endl;
```